CIS 4020 · DATA STRUCTURES

Recursion: Example Menu

Example solutions to practice with, in Java and C/C++

Every example here is a *problem solved with recursion*, given in both **Java** (CIS) and **C/C++** (CS). Use whichever examples help, you do not need to work through all of them. Each one names its **base case** and **recursive step** so you can find them quickly.

Table 1: Find an example by the recursion shape it demonstrates.

Example	Shape	Good for showing
Countdown	linear	the absolute basics; base case is "stop"
Factorial	linear	combine-on-the-backtrack
Sum of an array	linear over data	recursion on the arrays they know
Sum of an integer's digits	linear	reducing by `/10` and `%10`
Decimal to binary	linear, prints on return	order of operations in the backtrack
Palindrome check	linear, two ends	shrinking a range from both sides
Towers of Hanoi	tree-shaped	trusting recursion instead of tracing it all
Tribbles	linear growth	a real-world recurrence (and a great number)
Pokémon evolution	linear over data	walking a chain to a final form
Crafting cost	**tree**	branching recursion and why memoization matters
Flood fill	**2-D grid**	recursion on a grid; a bridge to the next topics
Even/odd raiders	**mutual**	indirect recursion (two methods calling each other)
Fractal detail	self-similar	a clean recurrence that mirrors the "mirrors" theme

1. Traditional examples

A1. Countdown: the simplest possible recursion

Print `n`, `n-1`, ..., `1`. *Base case:* `n == 0`, do nothing. *Recursive step:* print `n`, then count down from `n-1`.

```
void countdown(int n) {
    if (n == 0) return;              // base case
    System.out.println(n);
```

```
    countdown(n - 1);                // smaller
}
```

```
void countdown(int n) {
    if (n == 0) return;              // base case
    std::cout << n << "\n";
    countdown(n - 1);                // smaller
}
```

Tip. Try swapping the two lines (recurse *before* printing). It counts **up** instead of down. Ask yourself why: the print now happens on the backtrack.

A2. Factorial

Here $n! = n \cdot (n-1)!$. *Base:* `n <= 1` gives `1`. *Step:* `n * factorial(n-1)`.

```
long factorial(int n) { return (n <= 1) ? 1 : n * factorial(n - 1); }
```


```
long factorial(int n) { return (n <= 1) ? 1 : n * factorial(n - 1); }
```


A3. Sum of an array (ties to arrays they already know)

Sum `a[lo..hi]`. *Base:* `lo > hi` gives `0`. *Step:* `a[lo] + sum(rest)`.

```
int sum(int[] a, int lo, int hi) {
    if (lo > hi) return 0;                    // base: empty range
    return a[lo] + sum(a, lo + 1, hi);        // one fewer element
}
```

```
int sum(const std::vector<int>& a, int lo, int hi) {
    if (lo > hi) return 0;
    return a[lo] + sum(a, lo + 1, hi);
}
```

A4. Sum of an integer's digits

Here `sumDigits(1234) = 1+2+3+4`. *Base:* `n < 10` gives `n`. *Step:* `n % 10 + sumDigits(n / 10)`.

```
int sumDigits(int n) { return (n < 10) ? n : n % 10 + sumDigits(n / 10); }
```


```
int sumDigits(int n) { return (n < 10) ? n : n % 10 + sumDigits(n / 10); }
```


A5. Decimal to binary (the printing order is the lesson)

Base: `n < 2` prints `n`. *Step:* recurse on `n/2` **first**, then print `n % 2`.

```java
void toBinary(int n) {
    if (n < 2) { System.out.print(n); return; }  // base
    toBinary(n / 2);                              // recurse first
    System.out.print(n % 2);                      // then print on the backtrack
}
```

```cpp
void toBinary(int n) {
    if (n < 2) { std::cout << n; return; }
    toBinary(n / 2);
    std::cout << (n % 2);
}
```

> **Tip.** Predict the output of `toBinary(13)` before you run it. The bits come out most-significant-first precisely because the printing waits for the recursion to backtrack. This is the call-stack idea from the notes made visible.

A6. Palindrome check (shrink from both ends)

Base: a range of 0 or 1 characters is `true`; ends that differ are `false`. *Step:* if the ends match, check the inside.

```java
boolean isPalindrome(String s, int lo, int hi) {
    if (lo >= hi) return true;                    // base: nothing left to compare
    if (s.charAt(lo) != s.charAt(hi)) return false;
    return isPalindrome(s, lo + 1, hi - 1);       // smaller range
}
```

```cpp
bool isPalindrome(const std::string& s, int lo, int hi) {
    if (lo >= hi) return true;
    if (s[lo] != s[hi]) return false;
    return isPalindrome(s, lo + 1, hi - 1);
}
```

A7. Towers of Hanoi (the showpiece)

Move `n` disks from `from` to `to` using `via`. *Base:* `n == 0`, nothing to do.

```java
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);                  // top n-1 out of the way
    System.out.println(from + " -> " + to);       // move the big disk
    hanoi(n - 1, via, to, from);                  // bring them back on top
}
```

```cpp
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    std::cout << from << " -> " << to << "\n";
```

```
    hanoi(n - 1, via, to, from);
}
```

> **Tip.** Moving `n` disks takes $2^n - 1$ moves. Trace `n = 3` by hand, then *trust* it for `n = 10`. Learning to trust the recursion is the whole point of this one.

2. Pop-culture examples

Same recursion, nerdier packaging. These are real solutions to real (if silly) problems, and they deliberately cover different *shapes* so you can match one to whatever concept you are on.

B1. Tribbles: exponential growth (Star Trek)

In "The Trouble with Tribbles," a tribble produces an average litter of ten every twelve hours. So over each period, one tribble becomes eleven (itself plus ten). How many after `p` periods, starting from one? *Base:* `p == 0` gives `1`. *Step:* `11 * tribbles(p - 1)`.

```
long tribbles(int p) {
    if (p == 0) return 1;                          // base: the one you started with
    return 11 * tribbles(p - 1);                   // each period times 11
}
// tribbles(6)  ->  1,771,561   (three days of twelve-hour periods; the episode's
    number)
```

```
long tribbles(int p) {
    if (p == 0) return 1;
    return 11 * tribbles(p - 1);
}
// tribbles(6) == 1771561
```

> **Tip.** Landing on the show's actual number (1,771,561) is a nice payoff. It is a good example of growth that explodes, and a preview of why we care about efficiency.

B2. Pokémon evolution chain: walk a chain to its final form

An evolution chain is just an ordered list: `Charmander -> Charmeleon -> Charizard`. Recurse down the list until there is nothing left to evolve into. *Base:* at the last stage, return it. *Step:* move to the next stage.

```
// chain = ["Charmander", "Charmeleon", "Charizard"]
String finalForm(String[] chain, int i) {
    if (i == chain.length - 1) return chain[i];    // base: fully evolved
    return finalForm(chain, i + 1);                // evolve once more
}
```

```
std::string finalForm(const std::vector<std::string>& chain, int i) {
    if (i == chain.size() - 1) return chain[i];    // base: fully evolved
    return finalForm(chain, i + 1);
}
```

Tip. Try it yourself: write `countEvolutions(chain, i)`, returning how many steps to the final form. Same shape, but it returns a number instead of a string.

B3. Crafting cost: tree recursion (Minecraft or Factorio)

To craft an item you need ingredients; some of *those* must be crafted from still smaller ingredients. Compute the total **raw** materials needed. *Base:* a raw resource costs one of itself. *Step:* sum the raw cost of every ingredient, times how many you need. The branches make a tree.

```java
// recipes: item -> { ingredient -> quantity }. Missing item = raw resource.
int rawUnits(String item, Map<String, Map<String,Integer>> recipes) {
    if (!recipes.containsKey(item)) return 1;                    // base: raw resource
    int total = 0;
    for (Map.Entry<String,Integer> ing : recipes.get(item).entrySet())   // step: sum
    branches
        total += ing.getValue() * rawUnits(ing.getKey(), recipes);
    return total;
}
```

```cpp
// recipes: item -> { ingredient -> quantity }. Missing item = raw resource.
int rawUnits(const std::string& item,
             const std::map<std::string, std::map<std::string,int>>& recipes) {
    auto it = recipes.find(item);
    if (it == recipes.end()) return 1;                           // base: raw resource
    int total = 0;
    for (const auto& [name, qty] : it->second)                  // step: sum branches
        total += qty * rawUnits(name, recipes);
    return total;
}
```

Tip. This is the tree recursion from the slides in disguise. If two different recipes both need "iron ingot," you recompute its cost twice, which is exactly the Fibonacci problem. A natural place to think about **memoization**: cache each item's cost the first time you compute it.

B4. Flood fill: recursion on a 2-D grid (Minecraft "fill" bucket)

Replace a connected region of one tile with another: clear a lake, paint a cavern, spread water. From a starting cell, change it and recurse to its four neighbors. *Base (guards):* off the grid, or not the target tile, stop. This bridges the one-dimensional arrays they know to the grids coming next.

```java
void floodFill(char[][] g, int r, int c, char target, char fill) {
    if (r < 0 || r >= g.length || c < 0 || c >= g[0].length) return;  // base: off-grid
    if (g[r][c] != target) return;                                    // base: wrong
    tile
    g[r][c] = fill;                                                    // do the work
    floodFill(g, r + 1, c, target, fill);                             // four neighbors
    floodFill(g, r - 1, c, target, fill);
    floodFill(g, r, c + 1, target, fill);
    floodFill(g, r, c - 1, target, fill);
```

```
}
```

```
void floodFill(std::vector<std::vector<char>>& g, int r, int c, char target, char fill)
     {
    if (r < 0 || r >= (int)g.size() || c < 0 || c >= (int)g[0].size()) return; // off-
    grid
    if (g[r][c] != target) return;                                          // wrong
    tile
    g[r][c] = fill;
    floodFill(g, r + 1, c, target, fill);
    floodFill(g, r - 1, c, target, fill);
    floodFill(g, r, c + 1, target, fill);
    floodFill(g, r, c - 1, target, fill);
}
```

> **Tip.** Note the C++ grid is passed **by reference** (`&`): without it, every call copies the whole grid and nothing you write sticks. A classic C++ recursion trap; see my notes on C++ versus Java.

B5. Even or odd? Mutual recursion (Battlestar Galactica)

You want to know whether a Cylon raider count is even so the fleet can pair off. Two methods call *each other*: that is **indirect (mutual) recursion**, and it counts as recursion even though neither method calls itself. *Base:* zero is even and not odd. *Step:* peel one off and flip.

```
boolean isEven(int n) { return (n == 0) ?  true  : isOdd(n - 1); }
boolean isOdd (int n) { return (n == 0) ?  false : isEven(n - 1); }
```

```
bool isOdd(int n);                    // forward declaration: C++ must see it first
bool isEven(int n) { return (n == 0) ?  true  : isOdd(n - 1); }
bool isOdd (int n) { return (n == 0) ?  false : isEven(n - 1); }
```

> **Tip.** Wildly inefficient as a parity test (`n % 2` exists!), and that is fine: the point is to show recursion does not have to be a method calling *itself*. In C++, note the forward declaration, because `isEven` names `isOdd` before `isOdd` is defined, so the compiler needs the prototype first.

B6. Fractal detail: a self-similar recurrence (procedural worlds and the "mirrors" theme)

A Sierpiński triangle at each level is made of three smaller copies of itself, a shape reflected inside itself, like the mirror image on the title slide. How many solid triangles at detail level `n`? *Base:* `n == 0` gives `1`. *Step:* three copies of the level below.

```
long triangles(int n) {
    if (n == 0) return 1;                          // base: one solid triangle
    return 3 * triangles(n - 1);                   // three self-similar copies
}
// triangles(n) == 3^n; detail explodes as you zoom in
```

```
long triangles(int n) {
    if (n == 0) return 1;
    return 3 * triangles(n - 1);
}
```

> **Tip.** Ties the whole unit back to the opening image: a thing built out of smaller copies of itself. If you want to code one up, the recursive version subdivides a triangle into three and recurses on each, the same recurrence, now on screen.

A note. If any snippet does not compile or behaves oddly on your setup, tell me. These are written to be readable first, and I would rather fix a real bug than ship a clean-looking wrong one.